Compiler Design - Project 2

413410110 吳俊霆

Parser Features

- Core Functionality: The parser successfully performs syntax analysis for a specifically defined subset of the C programming language, utilizing ANTLR to generate the parsing rules.
- Grammar Tracing: To verify correct parsing, the grammar is instrumented to output specific grammar rules (via trace messages) as they are successfully matched during the parsing process.
- Supported Data Types: The parser supports variable and function types limited to int (integers), float (32-bit floating points), and void.
- Variables and Arrays: It supports the declaration of both global and local variables. It also successfully parses single-dimensional arrays with constant integer sizes (e.g., int arr[10];).
- Function Declarations: The parser handles function definitions that can take parameters (passed by value) and return `int`, `float`, or `void`. It specifically supports functions with no parameters by allowing the parentheses to be completely empty `( )` or by using the explicit `(void)` keyword. It requires a `main` function.
- Control Structures: The grammar accurately parses fundamental control flow statements, including if statements (with optional else branches), while loops, and return statements.
- Operators and Expressions: The parser supports a robust set of operations:
  - Arithmetic Operators: Addition (+), subtraction (-), multiplication (*), division (/), and modulo (%).
  - Relational Operators: Less than (<), greater than (>), less than or equal to (<=), greater than or equal to (>=), equal to (==), and not equal to (!=).
  - Assignment: The standard assignment operator (=).
- Comments: The lexical analyzer successfully identifies and skips both single-line comments (//) and multi-line block comments (/* ... */).

How to Compile and Execute

Step 1 - Build everything with make:

make
(Runs generate compile then compiles three test programs)

Step 1(optional) - Run individual tests:

make test1 # tests/test1.c
make test2 # tests/test2.c
make test3 # tests/test3.c

Step 2 - Clean files:

make clean

Trace Output

When TRACEON is true (the default), each matched grammar rule prints a one-line trace to stdout. For example, parsing int x; produces:
type: INT

declaration: type ID ;
declarations: declaration declarations
declarations: (empty)
...
At the end the driver prints either Parse SUCCESSFUL or Parse FAILED with an error count, and also displays the ANTLR parse tree.